Program Studi Teknik Informatika
Institut Teknologi Sumatera

IF25-22014 PENGEMBANGAN APLIKASI WEB

Modul Belajar 10

OOP di Python

Memodelkan data kampus sebagai object, bekal langsung untuk model FastAPI

Tim Pengajar

Pengembangan Aplikasi Web. Tahun 2026

Komponen	Keterangan
Pertemuan	10 dari 16
Capaian	Sub-CPMK 10.1 dan 10.2, turunan CPMK0502
Waktu belajar	sekitar 3 jam belajar mandiri, ditambah 150 menit di kelas
Hands-on	`02 Hands-on/P10 - OOP Python - Hands-on/`
Yang perlu disiapkan	Python 3.11 atau lebih baru dengan virtual environment dari P9

Peta belajar

Modul ini mengikuti bagian yang sama dengan slide. Setiap konsep inti ditulis dalam kotak gelap: itulah kalimat yang harus bisa kalian jelaskan ulang.

Bagian	Judul	Yang dipelajari
1	Pengantar OOP	Menjelaskan paradigma berorientasi object
2	Class dan object	Membuat class dan object di Python
3	Atribut dan method	Membedakan atribut object, atribut class, dan method
4	Konstruktor dan self	Menjelaskan peran __init__ dan self
5	Inheritance	Mewariskan atribut dan method ke class turunan
6	Encapsulation	Melindungi data dengan konvensi akses dan property
7	Polymorphism	Memanggil method yang sama pada object berbeda
8	Abstract class dan interface	Membuat template wajib untuk class turunan, dan mengenal model data modern

Tentang modul ini

Modul ini menemani Pertemuan 10. Setiap kotak merah **Tebak dulu** meminta kalian menulis dugaan sebelum membaca kotak hijau **Jawaban** di bawahnya. Kerjakan contoh kode sambil membaca, jangan hanya dibaca.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Diukur di
10.1	Menjelaskan class, object, atribut, method, konstruktor, dan self	Exit ticket soal 1, hands-on, proyek
10.2	Menerapkan inheritance, encapsulation, polymorphism, dan abstract class dalam model data	Exit ticket soal 2, hands-on, proyek

Keduanya menurunkan CPMK0502: mahasiswa mampu menjelaskan konsep pemrograman untuk pengembangan perangkat lunak.

Masalah pembuka

Dict yang tidak bisa menjaga dirinya

Data mahasiswa disimpan sebagai dict. Suatu hari seseorang menulis kode di samping, dan IPK seluruh angkatan menjadi kacau di laporan.

Dict menerima nilai apa pun dengan kunci apa pun, termasuk salah ketik.

TEBAK DULU

Bagaimana membuat data mahasiswa yang menolak nilai tidak masuk akal dan salah ketik nama kunci?

```
mhs = {"nim": "124140127", "nama": "Rani"}
mhs["nilai"] = 150 # tidak masuk akal
mhs["nialai"] = 80 # salah ketik
mhs["sks"] = "dua puluh" # tipe salah
print(mhs) # semua diterima
```

Bagian 1: Pengantar OOP

TUJUAN: MENJELASKAN PARADIGMA BERORIENTASI OBJECT

Data dan perilaku dalam satu tempat

Pemrograman berorientasi object (OOP)

Paradigma pemrograman yang memodelkan program sebagai kumpulan object; setiap object menyatukan data (atribut) dan perilaku (method).

Dengan dict, data mahasiswa dan fungsi `ipk` terpisah dan siapa pun bisa mengubah data seenaknya. Dengan OOP,

mahasiswa adalah object yang tahu cara menghitung IPK-nya sendiri dan menolak data yang salah. Empat pilar yang dibahas hari ini: encapsulation, inheritance, polymorphism, dan abstraction.

Inti Bagian 1

OOP menyatukan data (atribut) dan perilaku (method) dalam object.

Bagian 2: Class dan object

TUJUAN: MEMBUAT CLASS DAN OBJECT DI PYTHON

Class di JavaScript dan Python

JavaScript (P6)

```
class Mahasiswa {
  constructor(nim, nama) {
    this.nim = nim;
    this.nama = nama;
  }
  sapa() { return `Halo ${this.nama}`; }
}
const r = new Mahasiswa("127", "Rani");
```

Python

```
class Mahasiswa:
    def __init__(self, nim, nama):
        self.nim = nim
        self.nama = nama

    def sapa(self):
        return f"Halo {self.nama}"

r = Mahasiswa("127", "Rani")
```

Idenya sama dengan P6. Bedanya: `__init__` bukan `constructor`, `self` ditulis eksplisit sebagai parameter pertama, dan tidak ada kata `new`.

Ikuti bersama: class pertama di Python interaktif

Kerjakan sambil membaca, satu langkah demi satu langkah.

No	Langkah	Perintah atau kode
1	Buka Python interaktif di terminal.	`python`
2	Ketik kepala class, lalu Enter.	`class Mhs:`
3	Ketik konstruktor dengan empat spasi di depan, lalu Enter dua kali.	`def __init__(self, n): self.nama = n`
4	Buat object dan baca atributnya.	`r = Mhs("Rani"); r.nama`
5	Coba buat method tanpa `self`, lalu panggil.	

YANG HARUS TERLIHAT

Python menampilkan `'Rani'`. Method tanpa `self` memunculkan TypeError yang menyebut jumlah argumen.

Kalau berbeda

IndentationError: setiap baris di dalam class berawal empat spasi. NameError: nama class harus ditulis sama persis, termasuk huruf besar.

Inti Bagian 2

class cetakan, object hasilnya; setiap object punya data sendiri.

Bagian 3: Atribut dan method

TUJUAN: MEMBEDAKAN ATRIBUT OBJECT, ATRIBUT CLASS, DAN METHOD

Atribut dan method

```
class Mahasiswa:
    kampus = "ITERA" # atribut class

    def __init__(self, nim: str, nama: str):
        self.nim = nim # atribut object
        self.nama = nama
        self.khs: list[dict] = []

    def tambah_nilai(self, matkul: str, sks: int, huruf: str):
        self.khs.append({"matkul": matkul, "sks": sks, "huruf": huruf})

    def total_sks(self) -> int:
        return sum(m["sks"] for m in self.khs)

r = Mahasiswa("124140127", "Rani")
r.tambah_nilai("Pemweb", 3, "A"); print(r.total_sks())
```

Atribut class `kampus`: satu nilai untuk semua mahasiswa.

Atribut object `nim`, `nama`, `khs`: setiap mahasiswa punya sendiri. Jangan menaruh list sebagai atribut class, karena akan terbagi ke semua object.

Method: fungsi di dalam class; mengubah atau membaca data object lewat `self`.

Akses dengan titik: `r.total_sks()`. Bandingkan dengan dict yang memakai kurung siku.

Inti Bagian 3

Atribut object milik sendiri, atribut class dibagi; method bekerja pada self.

Bagian 4: Konstruktor dan self

TUJUAN: MENJELASKAN PERAN `__INIT__` DAN SELF

Siapa self?

TEBAK DULU

Saat menulis `r.total_sks()`, method `total_sks(self)` dipanggil tanpa argumen. Dari mana `self` datang, dan apa isinya?

JAWABAN

Python mengubah `r.total_sks()` menjadi `Mahasiswa.total_sks(r)`. Jadi `self` adalah `r`, object yang sedang dipakai. Itu sebabnya `self` wajib ditulis sebagai parameter pertama setiap method, dan atribut selalu diakses lewat `self.nama`, bukan `nama` saja.

Validasi di konstruktor

```
class Mahasiswa:
    def __init__(self, nim: str, nama: str):
        if not (nim.isdigit() and len(nim) == 9):
            raise ValueError("NIM harus 9 digit angka")
        if not nama.strip():
            raise ValueError("Nama tidak boleh kosong")
        self.nim = nim
        self.nama = nama.strip()

Mahasiswa("12AB", "Rani")
# ValueError: NIM harus 9 digit angka
```

`__init__` adalah gerbang: object yang lolos dijamin datanya benar.

`raise ValueError(...)`: tolak dengan pesan yang menyebut cara memperbaiki, pedoman pesan error P4.

Inilah bagian pertama jawaban masalah minggu ini. FastAPI di P11 melakukan validasi semacam ini otomatis dari type hints.

Inti Bagian 4

__init__ dijalankan saat object dibuat untuk validasi awal; self object yang sedang dipakai.

Bagian 5: Inheritance

TUJUAN: MEWARISKAN ATRIBUT DAN METHOD KE CLASS TURUNAN

Pewarisan

```
class Pengguna:
    def __init__(self, nama: str, email: str):
        self.nama = nama
        self.email = email

    def menu(self) -> list[str]:
        return ["Beranda"]

class Mahasiswa(Pengguna):
    def __init__(self, nama: str, email: str, nim: str):
        super().__init__(nama, email)
        self.nim = nim

    def menu(self) -> list[str]:
        return super().menu() + ["KRS", "KHS"]
```

`class Mahasiswa(Pengguna)`: Mahasiswa adalah Pengguna, setara `extends` di P6.

`super().__init__(...)`: biarkan induk mengisi atribut miliknya, lalu tambah milik sendiri.

Method boleh ditimpa (overriding), dan tetap bisa memakai versi induk lewat `super()`.

Di P11 kalian menulis `class Matkul(BaseModel)`: mewarisi kemampuan validasi dari Pydantic.

Inti Bagian 5

class Anak(Induk) mewarisi atribut dan method; super() memanggil induk.

Bagian 6: Encapsulation

TUJUAN: MELINDUNGI DATA DENGAN KONVENSI AKSES DAN PROPERTY

Konvensi akses dan property

```
class NilaiMatkul:
    def __init__(self, matkul: str, nilai: float):
        self.matkul = matkul
        self.nilai = nilai # lewat setter di bawah

    @property
    def nilai(self) → float:
        return self._nilai

    @nilai.setter
    def nilai(self, n: float):
        if not 0 ⩽ n ⩽ 100:
            raise ValueError("Nilai harus 0 sampai 100")
        self._nilai = n

n = NilaiMatkul("Pemweb", 86)
n.nilai = 150 # ValueError
```

Python tidak punya `private` seperti Java. Konvensinya: `_nilai` berarti internal, jangan disentuh dari luar; `__nilai` mengganti nama untuk mencegah tabrakan.

`@property`: dibaca seperti atribut, `n.nilai`, tetapi sebenarnya method.

`@nilai.setter`: setiap penugasan `n.nilai = ...` lewat pemeriksaan, termasuk yang di konstruktor.

Bagian kedua jawaban masalah minggu ini: nilai 150 ditolak.

Inti Bagian 6

Garis bawah menandai internal; @property dan setter menjaga nilai.

Bagian 7: Polymorphism

TUJUAN: MEMANGGIL METHOD YANG SAMA PADA OBJECT BERBEDA

Satu panggilan, banyak perilaku



Perulangan hanya memanggil p.menu(); setiap jenis pengguna menjawab dengan menunya sendiri. Ini jawaban misi 3. Versi bergeraknya ada di slide PowerPoint.

Hasil overriding

TEBAK DULU

Dengan class di animasi tadi, apa hasil `Pengguna("Tamu").menu()` dan `isinstance(Mahasiswa("R"), Pengguna)`? (A) Beranda dan False; (B) Beranda dan True; (C) KRS dan True; (D) error dan True

JAWABAN: B

Pengguna memakai method miliknya sendiri. Mahasiswa adalah turunan Pengguna, jadi `isinstance` bernilai True.

Inti Bagian 7

Method sama, perilaku berbeda per object; pemanggil tak perlu tahu jenisnya.

Bagian 8: Abstract class dan interface

TUJUAN: MEMBUAT TEMPLATE WAJIB UNTUK CLASS TURUNAN, DAN MENGENAL MODEL DATA MODERN

Abstract class

```
from abc import ABC, abstractmethod

class Penilaian(ABC):
    def __init__(self, bobot: float):
        self.bobot = bobot

    @abstractmethod
    def skor(self) → float: ...
class Kuis(Penilaian):
    def __init__(self, bobot: float, benar: int, total: int):
        super().__init__(bobot)
        self.benar, self.total = benar, total
    def skor(self) → float:
        return 100 * self.benar / self.total

Penilaian(0.2) # TypeError: tidak bisa dibuat langsung
```

Abstract class: cetakan setengah jadi yang tidak bisa dibuat langsung.

`@abstractmethod`: setiap turunan wajib menulis `skor()`. Lupa menulisnya berarti error saat object dibuat, bukan saat dipakai.

Python tidak punya kata `interface`; abstract class tanpa isi atau `typing.Protocol` dipakai sebagai interface.

Kode yang memakai `Penilaian` cukup memanggil `skor()` untuk kuis, tugas, atau ujian.

Class data siap pakai

dataclass (bawaan Python)

```
from dataclasses import dataclass

@dataclass
class Matkul:
    kode: str
    nama: str
    sks: int = 3

m = Matkul("IF25-22014", "Pemweb")
```

Pydantic, dipakai FastAPI

```
from pydantic import BaseModel, Field

class Matkul(BaseModel):
    kode: str
    nama: str
    sks: int = Field(3, ge=1, le=6)

Matkul(kode="IF", nama="Web", sks="dua")
# ValidationError: sks harus int
```

Pydantic menambah validasi dan konversi JSON pada class data: jawaban lengkap masalah minggu ini, dan model setiap endpoint P11.

Tiga baris yang salah

Yang ditulis	Mengapa salah, dan perbaikannya
`def total_sks():` di dalam class	Lupa `self`; TypeError saat dipanggil. Tulis `def total_sks(self):`.
`khs = []` sebagai atribut class	List terbagi ke semua object; nilai Rani muncul di KHS Bima. Pindahkan ke `self.khs = []` di `__init__`.
`super.__init__(nama)`	`super` harus dipanggil: `super().__init__(nama)`.

Pola kesalahannya

Kesalahan kedua paling berbahaya karena tidak ada error sama sekali; datanya diam-diam tercampur.

Giliran kalian

SETENGAH JADI: TUGAS SEBAGAI PENILAIAN

Lengkapi: `class Tugas(_____):` dengan `def skor(____) → float: return self._____` untuk nilai tugas 0 sampai 100.

MANDIRI: DOSEN

Tulis `class Dosen(Pengguna)` dengan atribut `nip` 18 digit yang divalidasi di konstruktor, dan `menu()` yang menambah *Nilai* dan *Bimbingan* ke menu induk.

JAWABAN

Setengah jadi: Penilaian, self, nilai. Mandiri: super().__init__(nama, email); if not (nip.isdigit() and len(nip) == 18): raise ValueError(...); menu mengembalikan super().menu() + [”Nilai”, ”Bimbingan”].

Inti Bagian 8

abstractmethod memaksa turunan; dataclass dan Pydantic BaseModel class data siap pakai.

Latihan mandiri

Kerjakan lembar dan berkas awal di folder `02 Hands-on/P10 - OOP Python - Hands-on/`.

Tingkat	Waktu	Yang dilatih
Tingkat 1, Dasar	10’	class, konstruktor dengan validasi, atribut, dan method
Tingkat 2, Menengah	15’	encapsulation dengan property, inheritance, dan polymorphism
Tingkat 3, Tantangan	10’	abstract class dan perhitungan nilai akhir
Pengayaan	sisa	model Pydantic sebagai persiapan P11

Kesalahan yang sering muncul

Yang ditulis	Masalahnya	Obatnya
method tanpa `self`	TypeError saat dipanggil	`self` parameter pertama
list di atribut class	data tercampur antarobject	`self.x = []` di `__init__`
`super.__init__`	super tidak dipanggil	`super().__init__()`
validasi di luar class	object bisa dibuat salah	validasi di konstruktor atau setter
`nama` tanpa `self.`	NameError di dalam method	`self.nama`
membuat object dari abstract class	TypeError	buat turunannya

Rangkuman

No	Bagian	Inti
1	Pengantar OOP	OOP menyatukan data (atribut) dan perilaku (method) dalam object.
2	Class dan object	class cetakan, object hasilnya; setiap object punya data sendiri.
3	Atribut dan method	Atribut object milik sendiri, atribut class dibagi; method bekerja pada self.
4	Konstruktor dan self	__init__ dijalankan saat object dibuat untuk validasi awal; self object yang sedang dipakai.
5	Inheritance	class Anak(Induk) mewarisi atribut dan method; super() memanggil induk.
6	Encapsulation	Garis bawah menandai internal; @property dan setter menjaga nilai.
7	Polymorphism	Method sama, perilaku berbeda per object; pemanggil tak perlu tahu jenisnya.
8	Abstract class dan interface	abstractmethod memaksa turunan; dataclass dan Pydantic BaseModel class data siap pakai.

Cek pemahaman

Jawab tanpa membuka modul atau slide. Mengingat kembali tanpa melihat lebih menguatkan ingatan daripada membaca ulang.

No	Soal
1	Apa beda class dan object?
2	Apa isi `self` saat `r.sapa()` dipanggil?
3	Kapan memakai atribut class?
4	Apa gunanya `@property` dengan setter?
5	Apa itu polymorphism?
6	Mengapa model FastAPI mewarisi `BaseModel`?

KUNCI CEK PEMAHAMAN

(1) Class cetakan, object hasil cetakan dengan datanya sendiri. (2) Object r. (3) Untuk nilai yang sama bagi semua object dan tidak diubah per object. (4) Menjaga nilai lewat pemeriksaan setiap kali dibaca atau diubah, tetapi tetap dipakai seperti atribut. (5) Object berbeda menjawab panggilan method yang sama dengan caranya sendiri. (6) Untuk mendapat validasi dan konversi JSON dari Pydantic.

Bacaan lanjut

Python, *The Python Tutorial*, bab 9 Classes (`docs.python.org/3/tutorial/classes.html`), dan modul `abc` serta `dataclasses`. Pydantic, *Models* (`docs.pydantic.dev`). Algifari dkk., ITERA Press, 2024, bab OOP di Python.